بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

44 - Cache Usage

Introduction

- Cache size
- Mapping address to multiword cache block

Cache Size

- Total number of bits needed for a cache is a function of cache size and address size
  - Because cache includes both storage for data and tags
- Size of the block is one word, but normally it is several

Cache Size

- For the following situation:
  - 32-bit addresses
  - Direct-mapped cache
  - Cache size is 2^n blocks, so n bits are used for index
  - Block size is 2^m words (2^{m+2} bytes), so m bits are used for word within the block, and two bits are used for the byte part of the address
- Size of tag field is

$$32 - (n + m + 2)$$

Cache Size

- Total number of bits in a direct-mapped cache is

$$2^n \times (\text{block size} + \text{tag size} + \text{valid field size})$$

- Since block size is 2^m words (2^{m+5} bits), and we need 1 bit for valid field, number of bits in such a cache is

$$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$

Cache Size

- Although this is the actual size in bits, the naming convention is to exclude size of the tag and valid field and to count only size of the data
  - Cache in the previous figure is called a 4 KiB cache

Example

- How many total bits are required for a direct-mapped cache with 16 KiB of data, 4-word blocks, and a 32-bit address?

Example (cont.)

- Number of words for 16 KiB data is 4096 (2^{12}) words.
  With block size of 4 words (2^2), there are 1024 (2^{10}) blocks.
  Each block has $4 \times 32 = 128$ bits of data plus a tag, which is
  32 - 10 - 2 - 2 bits, plus a valid bit.
  Total cache size is

$$2^{10} \times (4 \times 32 + (32 - 10 - 2 - 2) + 1) = 2^{10} \times 147 = 147 \text{ Kibibits}$$

  or 18.4 KiB for a 16 KiB cache.
  For this cache, the total number of bits in the cache is about
  1.15 times as many as needed just for the storage of the data.

Mapping Address to Multiword Cache Block

- Block number is given by

$$\text{(Block address) modulo (Number of blocks in the cache)}$$

where address of the block is

$$\frac{\text{Byte address}}{\text{Bytes per block}}$$

- This block address is block containing all addresses between

$$\left\lfloor \frac{\text{Byte address}}{\text{Bytes per block}} \right\rfloor \times \text{Bytes per block} \quad \text{and} \quad \left\lfloor \frac{\text{Byte address}}{\text{Bytes per block}} \right\rfloor \times \text{Bytes per block} + (\text{Bytes per block} - 1)$$

Example

- Consider a cache with 64 blocks and a block size of 16 bytes. To what block number does byte address 1200 map?
- With 16 bytes per block, byte address 1200 is block address

$$\left\lfloor \frac{1200}{6} \right\rfloor = 75$$

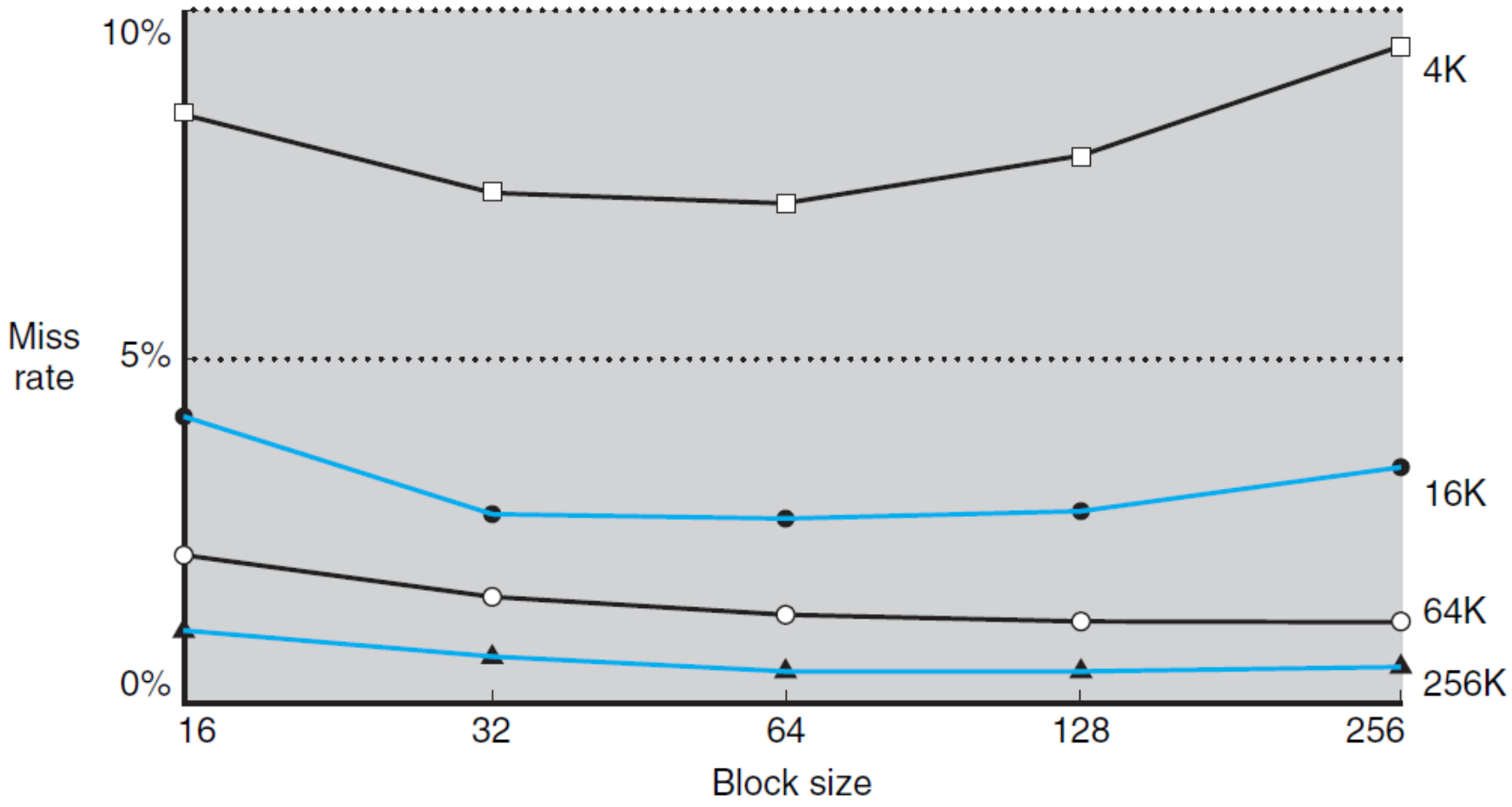
which maps to cache block number (75 modulo 64) 11

This block maps all addresses between 1200 and 1215.

Miss Rate vs Block Size

- Larger blocks exploit spatial locality to lower miss rate
  - Increasing block size usually decreases miss rate
  - Miss rate may go up eventually if the block size becomes a significant fraction of cache size
    - Because number of blocks that can be held in cache will become small
  - As a result, block will be bumped out of cache before many of its words are accessed
- Stated alternatively, spatial locality among the words in a block decreases with a very large block; consequently, benefits in the miss rate become smaller

Miss Rate vs Block Size



Miss Penalty and Block Size

- A more serious problem associated with just increasing block size is that the cost of a miss increases
- Miss penalty is determined by the time required to fetch block from next lower level of hierarchy and load it into cache
  - Time to fetch block has two parts
    - Latency to the first word and
    - Transfer time for the rest of block
- Transfer time, and hence miss penalty, will likely increase as the block size increases

Handling Cache Misses

- Control unit must detect a miss and process miss by fetching requested data from memory (a lower-level cache)
- If cache reports hit, computer continues using data as if nothing happened
- Cache miss handling is done in collaboration with processor control unit and with a separate controller that initiates the memory access and refills cache

Handling Cache Misses

- For a cache miss, we can stall the entire processor, essentially freezing contents of the temporary and programmer-visible registers, while we wait for memory
- More sophisticated out-of-order processors can allow execution of instructions while waiting for a cache miss
  - But we'll assume in-order processors that stall on cache misses

Handling Cache Misses

- If an instruction access results in a miss, then the content of the Instruction register is invalid
- To get the proper instruction into the cache, we must be able to instruct the lower level in the memory hierarchy to perform a read

Handling Instruction Cache Miss

1. Send original PC value (current PC – 4) to memory
2. Instruct main memory to perform a read and wait for the memory to complete its access
3. Write the cache entry, putting data from memory in the data portion of entry, writing upper bits of address (from ALU) into tag field, and turning valid bit on
4. Restart instruction execution at first step, which will refetch instruction, this time finding it in cache

Handling Data Cache Miss

- Control of cache on a data access is identical
- On a miss, stall processor until the memory responds with data

Handling Writes

- Suppose on a store instruction, we wrote data into only the data cache
  - without changing main memory
- Then, after the *write* into cache, memory would have different value from that in cache
- In such a case, cache and memory are said to be *inconsistent*

Write-Through

- Simplest way to keep main memory and cache consistent is always to *write* data into both memory and cache
- Called write-through

Performance

- This design handles writes very simply, but it would not provide very good performance
  - With WT, every write causes data to be written to main memory
- These writes will take a long time, likely at least 100 processor clock cycles, and could slow down the processor considerably
  - For example, suppose 10% of instructions are stores
    - If CPI without cache misses is 1.0, spending 100 extra cycles on every write would lead to a CPI of 1.0 + 100 × 10% = 11
      - Reducing performance by more than a factor of 10

Write Buffer

- One solution to this problem is to use a write buffer
  - It stores the data while it is waiting to be written to memory
- After writing data into cache and into write buffer, processor can continue execution
- When a write to main memory completes, entry in write buffer is freed

Write Buffer

- If write buffer is full when processor reaches a write, processor must stall until there is an empty position in write buffer
- Of course, if rate at which memory can complete writes is less than rate at which processor is generating writes, no amount of buffering can help
  - Because writes are being generated faster than memory system can accept them

Write Buffer

- Rate at which writes are generated may also be *less* than rate at which memory can accept them, and yet stalls may still occur
  - Can happen when writes occur in bursts
- To reduce the occurrence of such stalls, processors usually increase the depth of the write buffer beyond a single entry

Write-Back

- Alternative to a write-through is a scheme called write-back
  - When a *write* occurs, new value is written only to the block in cache
- Modified block is written to the lower level of hierarchy when it is replaced

Performance

- Write-back schemes can improve performance, especially when processors can generate writes as fast or faster than the writes can be handled by main memory
- Write-back scheme is, however, more complex to implement than write-through